

Devoir surveillé terminal

Durée 2h00
(Corrections)

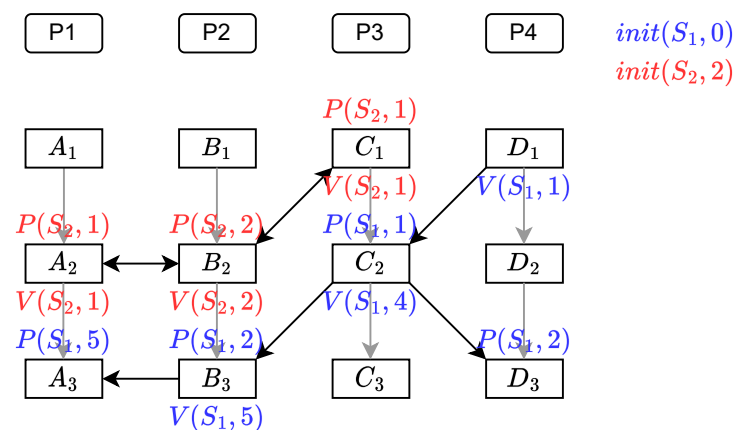
Exercice 1 (Questions de cours - 6 pts)

1°) Quelle(s) différence(s) et quelle(s) similarité(s) pouvez-vous donner entre une file de messages IPC System V et un tube nommé/anonyme ?

Solution : (2 points) Dans les deux cas, la lecture est destructive. La lecture/écriture dans une file implique l'utilisation d'un buffer, ce qui n'est pas le cas pour un tube. Les tubes utilisent des descripteurs de fichier, ce qui n'est pas le cas pour la file de messages. L'écriture dans une file est atomique, ce qui n'est pas le cas pour un tube. Dans les deux cas, la lecture et l'écriture sont bloquantes quand la file ou le tube est plein.

2°) Résolvez le diagramme de précedence suivant en utilisant 2 sémaphores généralisés.

Solution : (2 points)



3°) Donnez la portion de code permettant de créer et d'initialiser le tableau de sémaphores pour ce diagramme. La clé est fixée par la constante KEY. Si le tableau existe, un message d'erreur doit être affiché.

Solution : (2 point)

```
if((semid = semget(KEY, 2, S_IRUSR | S_IWUSR | IPC_CREAT | IPC_EXCL)) == -1) {
    if(errno == EEXIST) {
        fprintf(stderr, "Le_tableau_existe_déjà.\n");
        exit(EXIT_FAILURE);
    }
}
unsigned short values[2] = { 0, 2 };
semctl(semid, 0, SETALL, values);
```

Exercice 2 (Autorité de certification - 9 pts)

Nous souhaitons mettre en place le principe de certificats à l'aide de clés privées/publiques. Une application joue le rôle d'autorité de certification (nommée AdC dans la suite). Elle écoute sur deux ports différents : le port X pour la génération de certificats, utilisant des communications via TCP, et le port Y pour la récupération du certificat de l'AdC, utilisant des communications via UDP. Les numéros de port X et Y sont fixés comme constantes dans l'application.

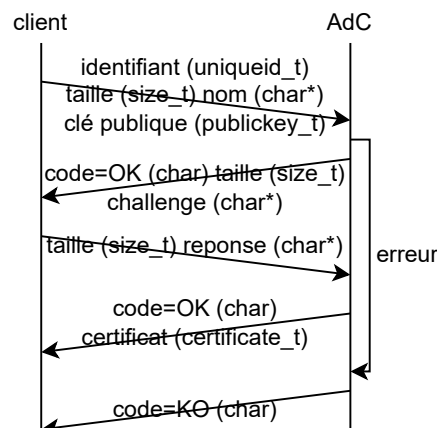
Pour générer un certificat, un client envoie ses informations à l'AdC : un identifiant unique généré à partir du matériel de la machine (de type `uniqueid_t`), un nom (une chaîne de longueur variable) et sa clé publique (de type `public_key_t`). Pour s'assurer que le client est bien le propriétaire de la clé publique, l'AdC envoie un message chiffré avec celle-ci et le client renvoie le message déchiffré. Si c'est correct, l'AdC envoie un certificat signé sa clé privée.

Lorsqu'un client *A* désire communiquer avec un client *B*, il envoie son certificat (de type `certificate_t`). Celui-ci peut être vérifié grâce au certificat de l'AdC que le client *B* peut récupérer à l'aide d'une requête vers l'AdC.

La génération de certificats

1°) Donnez le diagramme de communication entre le client et l'AdC. Précisez les données échangées et leur type, ainsi que la gestion d'erreur.

Solution : (1 point)



2°) Est-il nécessaire pour l'AdC de créer un fils pour chaque connexion avec un client ?

Solution : (0,5 point) *Le dialogue est assez court et le nombre de générations de certificat est sans doute assez faible, donc ce n'est pas nécessaire.*

3°) Donnez la portion de code de l'AdC permettant de gérer la génération de certificats. Elle doit contenir la création de la socket, la gestion des connexions avec des fils différents et la réception des informations du client. Les autres échanges ne sont pas à faire. La connexion doit être terminée proprement.

Solution : (4 points)

```

struct sockaddr_in address;
int fd, sockclient;

socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

memset(&address, 0, sizeof(struct sockaddr_in));
address.sin_family = AF_INET;
address.sin_addr.s_addr = htonl(INADDR_ANY);
address.sin_port = htons(PORT_X);

bind(fd, (struct sockaddr*)&address, sizeof(struct sockaddr_in));

listen(fd, 10);

while(stop == 0) {
    sockclient = accept(fd, NULL, NULL);

    if(fork() == 0) {
        close(fd);
    }
}

```

```

    uniqueid_t id;
    size_t size;
    char *name;
    public_key_t key;

    read(sockclient, &id, sizeof(id));
    read(sockclient, &size, sizeof(size));
    name = malloc(sizeof(size));
    read(sockclient, &name, size);
    read(sockclient, &key, sizeof(key));

    ...

    close(sockclient);
    exit(EXIT_SUCCESS);
}
else
    close(sockclient);
}
close(fd);

```

4°) Que peut-on dire sur la valeur spécifiée à `listen` ?

Solution : (0,5 point) Il s'agit du nombre maximum de connexions en attente. Au-delà de cette valeur, les connexions sont refusées. Comme un fils est créé à chaque connexion, la valeur peut être faible.

La récupération de certificats

5°) Proposez une solution pour que le serveur puisse récupérer l'adresse du client pour lui répondre.

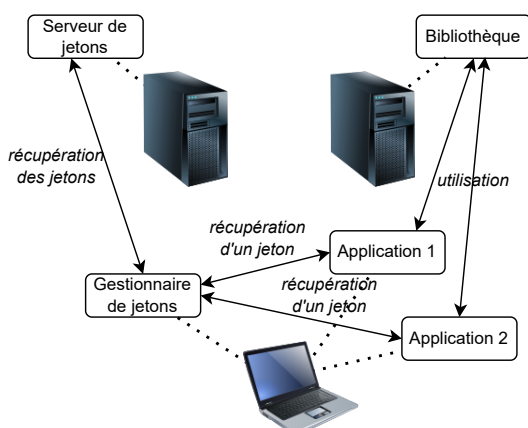
Solution : (1 point) Lors de la réception d'un message, avec `recvfrom`, il est possible de récupérer l'adresse du client. Il n'est pas nécessaire de mettre en place un autre mécanisme.

6°) Que se passe-t-il si le message du client ou la réponse du serveur sont perdus ? Proposez une solution. Précisez les appels systèmes nécessaires.

Solution : (2 points) On peut utiliser une alarme. Pour cela, on place un gestionnaire sur le signal `SIG_ALRM` (avec `sigaction`). On envoie le message (avec `sendto`) puis on place une alarme de 3 secondes (avec `alarm`), par exemple. Puis on attend la réponse (avec `recvfrom`). Si le message n'est pas reçu, la méthode est interrompue à la réception par l'alarme. Il est possible de le détecter grâce au retour de `recvfrom` et à la valeur de la variable `errno`.

Exercice 3 (Gestionnaire de jetons - 5 pts)

Nous supposons qu'une bibliothèque distante nécessite un jeton (une chaîne de caractères de 1024 octets) pour être utilisée. Sur une machine qui possède un abonnement, un nombre maximum d'applications peut accéder simultanément à cette bibliothèque.



Pour gérer localement, un gestionnaire récupère la liste des jetons auprès d'un serveur, le nombre dépendant de l'abonnement. Nous ne gérons pas les connexions dans cet exercice.

Le gestionnaire utilise un segment de mémoire partagé pour partager les jetons et leur état (occupé ou libre). Quand une application a besoin d'en utiliser un, elle le marque comme occupé puis se connecte à la bibliothèque (le jeton sert d'authentification). Elle le libère une fois qu'elle a terminée.

1°) Sachant que le nombre de jetons est variable, donnez le contenu du segment de mémoire partagé.

Solution : (0,5 point) Il contient le nombre de jetons (*int*), chaque jeton avec son état associé (*char[1024]* + *char* × 10).

2°) Proposez la structure `segment_t` permettant de mapper le segment.

Solution : (0,5 point)

```
typedef struct {
    int shmid;
    int *nb;
    struct jeton_t {
        char[1024] jeton;
        char etat;
    } *jetons;
} segment_t
```

3°) Donnez le code de la fonction `segment_t *mapping_segment(int key)` où `key` est la clé du segment de mémoire partagé. Elle permet de créer une structure pour mapper le segment.

Solution : (2 points)

```
segment_t *mapping_segment(int key) {
    segment_t *resultat = malloc(sizeof(segment_t));
    void *adr;

    resultat->shmid = shmget(key, 0, 0);
    adr = shmat(resultat->shmid, NULL, 0);
    resultat->nb = (int*)adr;
    resultat->jetons = (jeton_t*)&(resultat->nb[1]);

    return resultat;
}
```

4°) Quels sont les problèmes de synchronisation rencontrés ici quant à l'accès au segment de mémoire partagé ? Quelle solution proposez-vous ?

Solution : (1 point) Lorsqu'une application a besoin d'un jeton, elle doit consulter le segment de mémoire partagé en exclusion mutuelle. Elle cherche ensuite un jeton. S'il y en a un de disponible, il est marqué comme occupé. L'exclusion mutuelle est ensuite relâchée.

Nous souhaitons maintenant gérer le manque de jetons : quand tous les jetons sont occupés, une application demandeuse se met en attente.

5°) Proposez une solution à base de sémaphores en évitant l'attente active.

Solution : (1 point) On utilise un sémaphore initialisé avec le nombre de jetons. Quand une application souhaite un jeton, elle fait un P. Elle est donc bloquée s'il n'y a plus de jeton. Quand une application libère un jeton, elle fait un V. Une seule application peut alors capter le jeton libre.